# Track-System – Backend Files Map

Last updated: 17 Oct 2026 | Covers everything up to step 5 (React connection, password change, session, rate limit, tests) | What each file is for and how they are wired together | **[NEW]** = added in step 5

## 1. Where everything sits

```
Track-System/
  Frontend/ (React + Vite)
    .env.development            <- [NEW] VITE_API_URL (NOT in git: create it on each PC)
    src/
      main.jsx                  <- [NEW] starts the session keeper once (startSession)
      App.jsx                   <- the routes: /, /forgot-password, /change-password
      api/                      <- [NEW]
        client.js               <- [NEW] shared axios: API address, adds the token, 401 -> Login
        session.js              <- [NEW] refreshes the token while active, idle logout
      utils/                    <- [NEW]
        passwordRules.js        <- [NEW] the same password rules as the API
      pages/Login, ForgotPassword, ChangePassword   <- [NEW] Login + ChangePassword call the API
  Backend/
    TrackSystem.Api/
      TrackSystem.Api.csproj    <- list of libraries + UserSecretsId
      appsettings.json          <- settings (connection string, Jwt, [NEW] Cors, RateLimit)
      appsettings.Testing.json           <- [NEW] test settings (FinalProject_Test, limit 1000)
      appsettings.RateLimitTesting.json   <- [NEW] settings of the 429 test (limit 3)
      Program.cs                <- the server: setup + endpoints
      Data/
        AppDbContext.cs         <- the bridge to the database
      Models/                   <- C# classes that mirror tables
        Employee.cs             <- one row of dbo.Employees
        LoginLog.cs             <- one row of dbo.LoginLogs
      Dtos/                     <- shapes of data sent by the browser
        LoginRequest.cs         <- the login body (eNum + password)
        ChangePasswordRequest.cs <- [NEW] the change-password body
      Security/                 <- [NEW]
        PasswordRules.cs        <- [NEW] the rules for a new password
      login.http                <- local test requests (ignored by git)
      Docs/
        Backend-Files-Map.pdf   <- this file
        Backend-Glossary.pdf    <- the words to learn
      bin/ obj/ (build output - ignored by git)
    TrackSystem.Api.Tests/      <- [NEW] the automatic tests (xUnit)
      TrackSystem.Api.Tests.csproj  <- [NEW] test libraries + reference to TrackSystem.Api
      Security/
        PasswordRulesTests.cs   <- [NEW] unit tests (no API, no DB)
      Infrastructure/           <- [NEW] test helpers, not tests
        TestApiFactory.cs       <- [NEW] starts the API in memory + builds the test DB
        ApiCollection.cs        <- [NEW] one shared test API for all Api/ tests
        RateLimitApiFactory.cs  <- [NEW] a 2nd test API with limit 3
      Api/                      <- [NEW] integration tests, one file per area
        HealthTests.cs  LoginTests.cs  TokenTests.cs  EmployeesTests.cs
        ChangePasswordTests.cs  RefreshTests.cs  RateLimitTests.cs
      bin/ obj/ (build output - ignored by git)
  .gitignore (what git must not upload)

Outside the repo, on each PC:
  C:\Users\<you>\AppData\Roaming\Microsoft\UserSecrets\<id>\secrets.json
                              <- the JWT signing key (Jwt:Key)
  SQL Server (localhost\SQLEXPRESS):
    FinalProject                <- the real data
    FinalProject_Test           <- [NEW] made by dotnet test, dropped and rebuilt on every run
```

## 2. What each file does

| File | What it is | What it does | When do I touch it |
|---|---|---|---|
| TrackSystem.Api.csproj | Project definition | Lists the .NET version and every library: EF Core (SQL Server), BCrypt.Net-Next, JwtBearer, plus the UserSecretsId (the name of the secrets folder, not a secret itself). **[NEW]** Also System.Security.Cryptography.Pkcs 10.0.13: a direct reference that replaces a vulnerable version the SQL Server driver brought in indirectly. | Every time we add a library, or a vulnerability warning appears |

| File | What it is | What it does | When do I touch it |
|---|---|---|---|
| appsettings.json | Settings file (not code) | Connection string and the Jwt section (Issuer, Audience, ExpiresMinutes). The signing key is NOT here. **[NEW]** ExpiresMinutes is now 30 (sliding session). New sections: Cors:AllowedOrigins (the React address) and RateLimit:AuthPermitLimit (10 per minute). | When the server name, the DB or a token / CORS / limit setting changes |
| **[NEW]** appsettings.Testing.json | Test settings | Loaded on top of appsettings.json only in the environment "Testing" (the shared test API). Points to FinalProject_Test and raises the limit to 1000, so normal tests never get 429. In git (the partner needs it). | When the test DB or test limit changes |
| **[NEW]** appsettings.RateLimitTesting.json | Test settings | Same idea for the environment "RateLimitTesting": FinalProject_Test and a limit of 3. Used only by the 429 test. | Rarely |
| secrets.json (User Secrets) | Secret settings, outside the repo | Holds Jwt:Key, the key that signs and checks every token. Each PC has its own. Loaded only in Development. **[NEW]** Not used by the tests: they make their own random key. | Only with the dotnet user-secrets commands |
| Data/AppDbContext.cs | The bridge to the DB | Represents the database in C#. One DbSet line per table: Employees and LoginLogs. **[NEW]** The tests also use it to build FinalProject_Test from these models. | Every time we add a table |
| Models/Employee.cs | A model (a shape) | One row of dbo.Employees. The Password property holds a BCrypt hash, never the real password. | When a column in dbo.Employees changes |
| Models/LoginLog.cs | A model (a shape) | One row of dbo.LoginLogs: one login attempt (logID, eNum, loginDate, ipAddress, success). | When a column in dbo.LoginLogs changes |
| Dtos/LoginRequest.cs | A DTO (input shape) | What the browser sends to the login: eNum and password. ASP.NET turns the JSON body into this object. | If the login form sends other fields |
| **[NEW]** Dtos/ChangePasswordRequest.cs | A DTO (input shape) | What the change-password form sends: CurrentPassword and NewPassword. | If the form sends other fields |
| **[NEW]** Security/PasswordRules.cs | A static helper class | Validate(password) -> null if it passes, else the first broken rule as a message. Rules: 8-64 characters, one capital, one number, one special character. Used by change-password. | When a password rule changes (change passwordRules.js too) |
| Program.cs | The server | Setup area, then endpoints. **[NEW]** Setup now also registers CORS ("ReactApp"), the two access rules (default + "PasswordChange"), the rate limiter ("auth") and a dummy hash. Pipeline: UseCors -> UseRateLimiter -> UseAuthentication -> UseAuthorization. The local function CreateToken builds every token. New endpoints: change-password and refresh. | Every new endpoint or new service |
| .gitignore | Git rules | Stops build files, secrets and *.http files from being uploaded. **[NEW]** Its .env.* line also ignores Frontend/.env.development. | Rarely |
| login.http | Local test file (REST Client) | Ready-made requests for login and employees, **[NEW]** change-password, the RESET request and refresh. Not in git (it holds a test password). | When testing endpoints by hand |
| **[NEW]** Frontend/.env.development | Frontend settings | One line: VITE_API_URL=http://localhost:5163. Not in git, so each PC creates it by hand. | When the API address changes |
| **[NEW]** src/api/client.js | Shared axios | baseURL from .env.development. Before each request: adds Authorization: Bearer <token> from sessionStorage. After a 401 (not from login): removes the token and goes to "/". | Rarely; every page imports it to call the API |
| **[NEW]** src/api/session.js | Session keeper | On click / key / mouse / scroll: token older than 60 s -> POST /api/auth/refresh. Every 15 s: token expired -> removed, back to Login (idle logout). | When the session rules change |
| **[NEW]** src/utils/passwordRules.js | Frontend rules | The same rules and messages as PasswordRules.cs, so the user sees the error before sending. | Together with PasswordRules.cs |
| **[NEW]** src/main.jsx | Frontend start | Calls startSession() once, above createRoot. | Rarely |

| File | What it is | What it does | When do I touch it |
|---|---|---|---|
| **[NEW]** pages/Login, ChangePassword | React pages | Login: digits-only eNum, POST login, saves the token, mustChangePassword -> /change-password. ChangePassword: current / new / confirm, POST change-password, saves the new token. | When the page or its flow changes |
| **[NEW]** TrackSystem.Api.Tests.csproj | Test project definition | xUnit, the test runner, Microsoft.AspNetCore.Mvc.Testing, and a reference to TrackSystem.Api (so the tests see Program, AppDbContext, PasswordRules). EF Core, BCrypt and JwtBearer come through that reference. | When a test library is added |
| **[NEW]** Security/PasswordRulesTests.cs | Unit tests | Calls PasswordRules.Validate directly: a valid password, each broken rule, the 8 / 64 / 65 limits, null and empty. No API, no DB. | When a password rule changes |
| **[NEW]** Infrastructure/TestApiFactory.cs | Test helper (the factory) | Starts the API in memory in "Testing" with a random Jwt:Key. Once before the tests: drops and recreates FinalProject_Test (refuses any other DB name). Helpers: CreateEmployeeAsync, GetEmployeeAsync, GetLoginLogsAsync, LoginAsync, SetStatusAsync. | When tests need a new helper |
| **[NEW]** Infrastructure/ApiCollection.cs | Test helper | Collection "Api": one shared TestApiFactory for every class marked [Collection("Api")]. Those classes run one after another, never two at once on the test DB. | Rarely |
| **[NEW]** Infrastructure/RateLimitApiFactory.cs | Test helper | A second in-memory API in "RateLimitTesting" (limit 3) with its own counters, so the 429 test does not affect the shared API. | Rarely |
| **[NEW]** Api/*Tests.cs (7 files) | Integration tests | Real HTTP requests to the in-memory API, then checks of the status, the JSON body, the token claims and the DB rows. Health, Login, Token, Employees, ChangePassword, Refresh, RateLimit. | When an endpoint changes or a new one is added |

## 3. How the files are connected

```
Browser / React (port 5173)
   | [NEW] src/api/client.js: baseURL from .env.development + "Authorization: Bearer <token>"
   | [NEW] src/api/session.js: POST /api/auth/refresh while the user is active
   v
Program.cs (port 5163)
   | [NEW] UseCors: allows only the origin in ---------------> appsettings.json (Cors)
   | [NEW] UseRateLimiter: login + change-password, per IP -> appsettings.json (RateLimit)
   | UseAuthentication: checks the token with the key from --> secrets.json
   |                    and Issuer / Audience from ---------> appsettings.json
   | UseAuthorization: [NEW] default rule / "PasswordChange" -> 401 or 403
   |
   | the bodies arrive as -------------------> Dtos/LoginRequest.cs, [NEW] Dtos/ChangePasswordRequest.cs
   | [NEW] a new password is checked by ------> Security/PasswordRules.cs
   | Program.cs uses ------------------------> Data/AppDbContext.cs -> Models/Employee.cs, LoginLog.cs
   v
AppDbContext --(EF Core turns C# into SQL)--> SQL Server, DB FinalProject

[NEW] dotnet test:
TrackSystem.Api.Tests --(project reference)--> TrackSystem.Api
   TestApiFactory: starts Program.cs in memory, environment "Testing" -> appsettings.Testing.json
                   AppDbContext --> SQL Server, DB FinalProject_Test (dropped + created every run)
```

| This file | needs / uses | Why |
|---|---|---|
| Program.cs | appsettings.json | Reads DefaultConnection, Jwt:Issuer, Jwt:Audience, Jwt:ExpiresMinutes, **[NEW]** Cors:AllowedOrigins and RateLimit:AuthPermitLimit. |
| Program.cs | secrets.json (User Secrets) | Reads Jwt:Key. The same key signs tokens and checks them on every request. Without it the server refuses to start. |
| **[NEW]** Program.cs (CreateToken) | appsettings.json + secrets.json | One place that builds every token (login, change-password, refresh): sub, role, mustChangePassword, 30 min. |
| Program.cs (/api/auth/login) | Dtos/LoginRequest.cs | The login endpoint receives the body as a LoginRequest object. |

| This file | needs / uses | Why |
|---|---|---|
| Program.cs (/api/auth/login) | AppDbContext.Employees | Finds the employee, checks the hash with BCrypt, updates failedLoginAttempts / lockoutEnd. **[NEW]** Locked or not active -> the same generic 401. Unknown eNum or locked -> a dummy BCrypt check (equal timing). |
| Program.cs (/api/auth/login) | AppDbContext.LoginLogs | Adds one row per attempt. The row and the employee changes are saved together with SaveChangesAsync. |
| **[NEW]** Program.cs (/api/auth/change-password) | ChangePasswordRequest, PasswordRules, AppDbContext.Employees | eNum from the token's sub. Checks the current password, new != current, the rules; saves the new hash, IsFirstLogin = false, returns a new token. |
| **[NEW]** Program.cs (/api/auth/refresh) | AppDbContext.Employees | Re-reads the employee: deleted or not active -> 401, else a fresh 30-minute token. |
| Program.cs (/api/employees) | AppDbContext.Employees | Reads the employees and picks only the safe fields. Runs only with a valid token **[NEW]** whose mustChangePassword is "false". |
| AppDbContext.cs | Models/Employee.cs, Models/LoginLog.cs | Each DbSet line says which model shapes which table. |
| Models/LoginLog.cs | dbo.LoginLogs (the real table) | Property names must match the columns. eNum has no foreign key by design (unknown numbers are logged too). |
| **[NEW]** src/utils/passwordRules.js | Security/PasswordRules.cs | Same rules, same messages: change both together. |
| **[NEW]** src/api/session.js | src/api/client.js, /api/auth/refresh | Uses the shared axios, so a 401 from refresh also sends the user to Login. |
| **[NEW]** TrackSystem.Api.Tests | TrackSystem.Api (project reference) | The tests call the real classes and start the real Program.cs. |
| **[NEW]** TestApiFactory.cs | appsettings.Testing.json, AppDbContext | Environment "Testing" picks the test settings; EnsureCreated builds FinalProject_Test from the models. |
| Everything | TrackSystem.Api.csproj | If a library version does not match the .NET version, nothing builds. |

## 4. The addresses that exist today (all inside Program.cs)

| Address | Needs a token? | Answers | What it proves |
|---|---|---|---|
| GET /api/health | No | {"status":"ok"} | The server itself is alive |
| GET /api/db-check | No | {"database":"connected"} | The server reached the FinalProject database |
| POST /api/auth/login | No (this is where you get one). **[NEW]** Rate limit: 10 per minute per IP | 200: message, eNum, mustChangePassword, token (30 min). 401: "Invalid employee number or password" **[NEW]** also when locked or not active (no more 423). **[NEW]** 429 after 10 tries in a minute. | BCrypt check, 5-try lockout, logging of every attempt, **[NEW]** equal timing, token issued |
| GET /api/employees | Yes, the default rule | 200: the list (no passwords). 401: no, fake, edited or expired token. **[NEW]** 403: valid token with mustChangePassword "true". | The token guard and **[NEW]** the default rule work |
| **[NEW]** POST /api/auth/change-password | Yes, rule "PasswordChange" (also first-login tokens). Rate limit too | 200: message, mustChangePassword false, a new token. 400 + message: wrong current password, same as current, or a broken rule. 401: bad token. 429. | Forced first-login change, password rules |
| **[NEW]** POST /api/auth/refresh | Yes, rule "PasswordChange". No body, no rate limit | 200: token, mustChangePassword (read from the DB). 401: no token, or the employee is deleted / not active. | Sliding 30-minute session |

## 5. Rules for this project

| Rule | Why |
|---|---|
| Services are registered above var app = builder.Build(); | After the server is built, nothing can be registered any more. |
| After Build(), in this order: **[NEW]** UseCors, UseRateLimiter, then UseAuthentication, UseAuthorization, then the endpoints | The request pipeline runs top to bottom: CORS headers, count the attempt, identify the caller, apply the rules. |
| Server and DB names live in appsettings.json. Secret keys go to User Secrets. | One place to change, and nothing secret reaches the public GitHub repo. |
| All database traffic goes through AppDbContext | One single door to the DB. |
| The model may hold the password hash; an endpoint never sends it | Each endpoint chooses which fields leave the server. **[NEW]** A test checks the /api/employees answer has no hash. |
| Never put secrets in a token | The token payload is readable by anyone. Only IDs and flags go in it. |
| Times used by code are UTC; times read by people are Israel time | lockoutEnd is UTC (no daylight-saving jumps). LoginLogs.loginDate is Israel time. Sort logs by logID. |
| **[NEW]** 401 = token problem, 400 = wrong input, 403 = valid token but not allowed here, 429 = too many attempts | React reacts differently: 401 logs out, 400 shows the message, 403 keeps the user logged in. |
| **[NEW]** Password rules live in two places: PasswordRules.cs and passwordRules.js | The browser checks first for a quick message, the API checks again for safety. Change both. |
| A DB structure change is run on both machines and written in the Drive docs | The DB is not in git; each PC has its own copy. |
| **[NEW]** Tests use only FinalProject_Test, never FinalProject | The test setup drops a whole database. TestApiFactory stops if the name is anything else. |
| **[NEW]** Every test creates its own employee | Tests do not depend on each other's leftovers, and their order does not matter. |
| **[NEW]** dotnet test runs inside Backend\TrackSystem.Api.Tests. All green before every commit | A red test means something that worked before is broken. |
| After a code or appsettings change: save, stop (Ctrl+C), run again | The server reads both only at startup. |
| Run dotnet commands for the API inside Backend\TrackSystem.Api | From the repo root they fail with MSB1003. |
| bin/, obj/, secrets and *.http files stay out of git | Run git status before every commit. |
| **[NEW]** Frontend/.env.development is not in git | Each PC creates it: VITE_API_URL=http://localhost:5163. |

## 6. Open items

| Item | Details |
|---|---|
| **[NEW]** Closed in step 5 | mustChangePassword is enforced (403), rate limiting (429), equal response time (dummy hash), only "active" employees log in, CORS + React connected, xUnit test project created. |
| **[NEW]** Step 6: ForgotPassword | Email reset link: new table for one-time reset tokens (hash + expiry + used), MailKit + a local fake mailbox, a /reset-password page, its own tests. |
| **[NEW]** Main page after login | Login and ChangePassword still end with console.log (TODO). |
| /api/employees: any logged-in user | Needs a valid token (and a changed password), but any role may call it. Later: role rules. |
| **[NEW]** Old token after a password change | Stays valid until it expires (stateless JWT). Harmless: it still says "true", so it opens only change-password and refresh. |
| DB changes not tracked in git | Planned Database/changes/ folder with .sql files (e.g. dropping the LoginLogs FK). |
| Shared server name | If a PC's SQL instance has another name, override DefaultConnection in User Secrets on that PC only. **[NEW]** The test settings files use localhost\SQLEXPRESS too. |

| Item | Details |
| --- | --- |
| /api/db-check is open | A development tool; remove or protect it before any real deployment. |
| Files not created yet | Models for the other 8 tables, endpoint files (to keep Program.cs short), output DTOs, a Services layer for the login logic. |